

AMANE — FRAMEWORK SAAS SOUVERAIN

Organisation du dépôt Git et du développement

Structure du repo, stratégie de branches, gestion du contrat d'interface et intégration continue pour une équipe de 3 personnes

Stage R&D — 4 août 2026

1. Objet du document

Ce document définit comment organiser le développement d'Amane sur GitHub pour une équipe de 3 personnes travaillant sur 2 langages (Rust pour les missions A et B, Go pour la mission C). Il couvre la structure du repo, la stratégie de branches, la gestion du contrat d'interface partagé (.proto), et l'intégration continue qui fait respecter les contrats entre missions.

2. Un dépôt unique (monorepo) plutôt que 3 dépôts séparés

Malgré les deux langages, un monorepo est préférable à ce stade du projet. Les interfaces entre les missions A, B et C sont contractuelles et évoluent souvent (en particulier le contrat gRPC et le format du journal chiffré) : les garder dans un seul dépôt évite de synchroniser des versions entre plusieurs dépôts pendant une phase où le contrat n'est pas encore stable.

- Un seul historique Git pour tout le projet — plus simple pour retrouver quand une interface a changé et pourquoi.
- Une Pull Request peut toucher plusieurs missions à la fois si nécessaire (ex. un changement de .proto qui impacte B et C ensemble).
- Un seul pipeline de CI à maintenir, capable de construire Rust et Go dans la même exécution.

2.1 Structure de dossiers proposée

```
amane/  
├── core-rust/                # Missions A + B – crates Rust  
│   ├── ss-crypto/           # AK/DEK, KEK, Shamir, CRL (Mission A)  
│   ├── ss-journal/          # journal CBOR append-only  
│   └── ss-client/            # logique métier + SDK gRPC (Mission B)  
├── orchestrator-go/         # Mission C – service Go  
│   ├── consensus/           # intégration etcd + Patroni  
│   ├── replication/         # CRDT multi-site  
│   ├── mesh/                # WireGuard  
│   └── grpcserver/          # serveur gRPC exposant le framework  
├── proto/                   # définitions .proto – contrat partagé  
├── sdk-generated/           # SDK auto-générés (Python, JS...) pour éditeurs  
├── infra/                   # Docker Compose, configs de déploiement  
├── docs/                    # ADRs, diagrammes, contrats d'interface  
├── tests/                   # tests d'intégration cross-missions  
└── .github/workflows/       # CI
```

Le dossier proto/ mérite un traitement particulier : c'est le seul endroit du repo où les 3 missions se rencontrent réellement en code, pas seulement en documentation. Toute PR qui le touche est traitée comme sensible (voir section 4).

3. Stratégie de branches : trunk-based, branches courtes

Plutôt que 3 grandes branches "mission-a", "mission-b", "mission-c" qui vivraient plusieurs semaines et divergeraient, l'approche recommandée est un modèle trunk-based avec des branches courtes par tâche.

3.1 Principe

Élément	Règle
Branche principale	main — toujours stable, ce qui tourne réellement. Jamais de commit direct dessus.
Branches de travail	Courtes (quelques jours), préfixées par mission : mission-a/kek-hierarchy, mission-b/offline-read, mission-c/etcd-integration.
Fusion vers main	Uniquement via Pull Request, jamais de merge direct.
Durée de vie	Une branche qui dépasse une semaine sans PR est un signal à se demander si la tâche doit être découpée en plus petites étapes.

3.2 Pourquoi pas de longues branches par mission

Avec deux langages différents, un merge tardif est plus risqué qu'avec un seul langage : les divergences ne sont pas seulement du code métier mais potentiellement des changements de contrat d'interface non détectés à temps. Des branches courtes et des PR fréquentes permettent de repérer un désaccord entre deux missions en quelques jours plutôt qu'en fin de sprint.

3.3 Revue croisée

Une PR qui touche uniquement le périmètre interne d'une mission (ex. un détail d'implémentation dans ss-crypto qui ne change pas l'interface publique) peut être revue par la même mission. Une PR qui touche une interface partagée (proto/, ou tout point de contact décrit dans les contrats d'interface) doit être revue par au moins une personne de la mission en face — c'est elle qui valide que le contrat est respecté de son côté.

4. Le contrat .proto comme verrou de synchronisation

Le fichier .proto est le point de rupture le plus probable entre les missions B (qui consomme le SDK généré) et C (qui expose le service gRPC). Il est traité comme la ressource la plus sensible du repo.

1. Toute modification de proto/ passe par une PR séparée, jamais mélangée avec du code métier ou d'infrastructure.
2. La PR nécessite une revue par au moins une personne de chaque mission consommatrice du contrat (a minima B et C).
3. La CI régénère automatiquement les SDK (sdk-generated/) à partir du .proto modifié et vérifie que la génération réussit sans erreur avant tout merge.
4. Une fois mergé, le changement de contrat est communiqué explicitement à l'équipe — un changement de contrat silencieux est le scénario à éviter en priorité.

5. Intégration continue : faire respecter les contrats d'interface

Les 3 contrats d'interface définis dans le cadrage (Enrôlement $A \leftrightarrow C$, Frontière de chiffrement $A \leftrightarrow B$, Chemin d'écriture $B \leftrightarrow C$) sont traduits en tests d'intégration automatiques plutôt que laissés à la seule documentation.

Pipeline CI	Déclenchement	Vérifie
build-rust	Toute PR touchant core-rust/	Compilation des crates Rust, tests unitaires ss-crypto et ss-journal.
build-go	Toute PR touchant orchestrator-go/	Compilation du service Go, tests unitaires consensus et replication.

Pipeline CI	Déclenchement	Vérifie
proto-check	Toute PR touchant proto/	Régénération des SDK, absence de breaking change non documenté.
integration-tests	Toute PR touchant plusieurs dossiers, ou avant merge sur main	Les 3 contrats d'interface : aucun clair transmis au relais ($A \leftrightarrow B$), recalcul de quorum au dé-enrôlement ($A \leftrightarrow C$), confirmation d'écriture après réplication ($B \leftrightarrow C$).

6. Jalons de développement

Les jalons reprennent l'ordre du tableau des écarts identifiés entre le prototype existant et l'architecture cible. Le point 2 est particulièrement important à traiter tôt : tant que le contrat .proto n'existe pas, même sous une forme minimale, les missions B et C ne peuvent pas avancer réellement en parallèle sans risquer de tout refaire.

- Double clé AK/DEK + CRL (Mission A) — fondation qui bloque le reste du travail crypto.
- Contrat .proto initial + serveur gRPC squelette — débloque B et C en parallèle.
- Intégration etcd + Patroni (Mission C).
- Réseau maillé WireGuard (Mission C).
- Réplication CRDT multi-site (Mission C).
- Shamir SSS + récupération à 3 couches (Mission A).

Chaque jalon correspond à un GitHub Milestone regroupant ses issues et ses branches ; il se clôt par la PR de fusion finale vers main une fois les tests d'intégration associés au vert.

7. Synthèse

- Un seul dépôt (monorepo), avec un dossier par mission et un dossier proto/ partagé traité comme ressource sensible.
- Branches courtes par tâche, préfixées par mission, fusionnées fréquemment via Pull Request — pas de longues branches qui divergent.
- Revue croisée obligatoire sur toute PR touchant une interface partagée, en particulier proto/.
- CI qui compile Rust et Go séparément, régénère les SDK, et fait tourner des tests d'intégration qui vérifient concrètement les 3 contrats d'interface.
- Jalons alignés sur les écarts identifiés, avec le contrat .proto initial comme priorité pour débloquer le travail parallèle de B et C.